# Model Selection

DSC 340: Applied Machine Learning

Topics: Grid Search, Pipeline, Occam's Razor, Curse of Dimensionality

Week 7, Day 3, March 31

# Plan

- How to choose a best model
  - Grid search
  - sklearn Pipeline tool
- Occam's razor: Why less is often more
  - Usefulness of a model
  - Bias-variance tradeoff
  - Curse of dimensionality

# Searching for the Best Model

# Searching for the best model

- There are many **methods** for modeling classification problems:
  - logistic regression
  - kNN
  - decision tree
  - random forests
  - perceptron
  - *neural nets*
  - *naïve Bayes*
  - SVMs
  - C, penalty (L1, L2, etc.), solver, OvR/Multi
  - k, distance metric, weights on distance
  - gini/entropy, random/best split, max-depth
  - size of forest, # features/rows in split
  - *layout of network, activation function*
  - *smoothing parameter* $\lambda$
  - C, choice of kernel, degree/coef. (of poly)
- For each method, there are many **hyperparameters** we can select!

# Multiclass classification

- **One-versus-the-rest (OvR)** classification
  - $K$ models (one per class)
  - Can result in **ambiguous regions**
- **One-versus-one (OvO)** classification
  - $K(K-1)/2$ models (one per pair of classes)
  - Can result in **ambiguous regions**

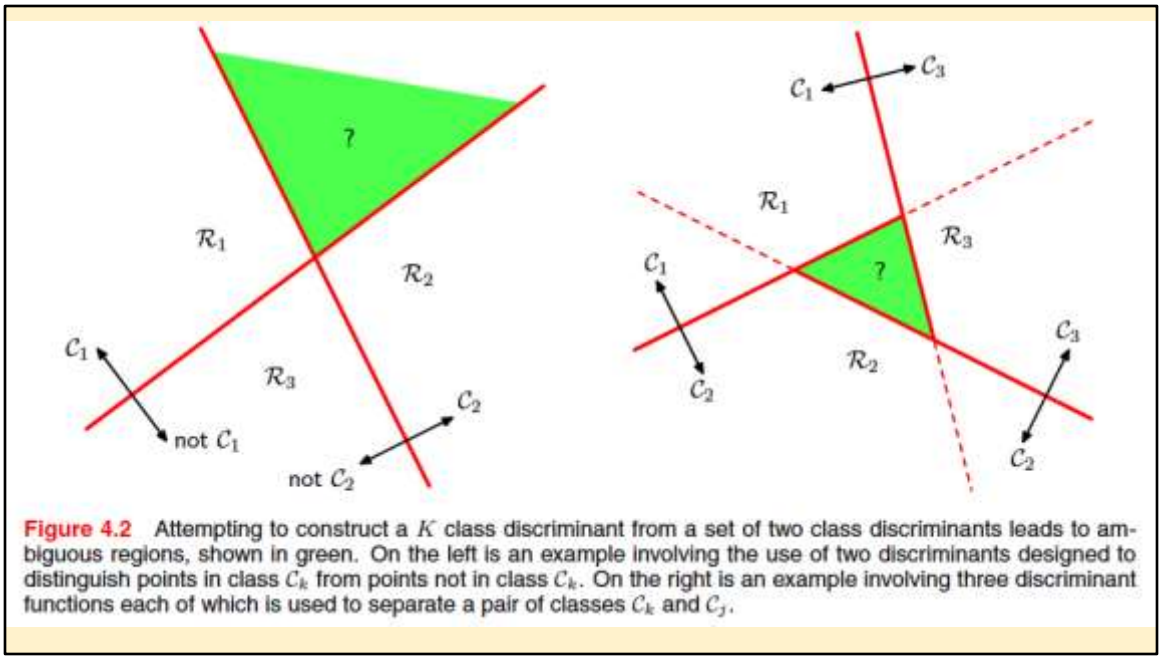



**Figure 4.2** Attempting to construct a $K$ class discriminant from a set of two class discriminants leads to ambiguous regions, shown in green. On the left is an example involving the use of two discriminants designed to distinguish points in class $\mathcal{C}_k$ from points not in class $\mathcal{C}_k$. On the right is an example involving three discriminant functions each of which is used to separate a pair of classes $\mathcal{C}_k$ and $\mathcal{C}_j$.

# Softmax (true multiclass classification)

- Softmax function:

$$g_k(\mathbf{x}, \mathbf{w}_k) = \frac{\exp(z_k)}{\sum_{j=0}^{t-1} \exp(z_j)}, \quad z_k = \mathbf{w}_k^\top \mathbf{x}, \tag{22}$$

- Provides smooth probability distribution across classes
- Sum of predicted probabilities equals 1 for a given instance

# Searching for the best model

- There are many **methods** for modeling classification problems:
  - logistic regression
  - kNN
  - decision tree
  - random forests
  - perceptron
  - *neural nets*
  - *naïve Bayes*
  - SVMs
  - C, penalty (L1, L2, etc.), solver, OvR/Multi
  - k, distance metric, weights on distance
  - gini/entropy, random/best split, max-depth
  - size of forest, # features/rows in split
  - *layout of network, activation function*
  - *smoothing parameter* $\lambda$
  - C, choice of kernel, degree/coef. (of poly)
- For each method, there are many **hyperparameters** we can select!

# Grid search

- Suppose 1 method and 2 hyperparameters with several values
  - Example: kNN: k = {1, 3, 5, 7}, distance metric = {L1:Manh., L2:Euclid.}
- I could write a **nested loop:**

```
for k in [1, 3, 5, 7]:
  for metric in ["manhattan", "euclidean"]:
    classifier = KNeighborsClassifier(n_neighbors=k, metric=metric)
    # evaluate model and store somehow
# pick best
```

- Known as **grid search**
  - Metaphor: spreadsheet with columns and rows (and sheets if 3D)

# Grid search example: kNN

| *accuracy score* | k=1 | k=3 | k=5 | k=7 |
|---|---|---|---|---|
| **Manhattan (L1)** | **0.83** | **0.91** | **0.95** | **0.89** |
| **Euclidean (L2)** | **0.80** | **0.79** | **0.92** | **0.90** |

- Try several hyperparameter combinations (rarely "all"!)
- Evaluate each on validation set
- Choose one with best score: highest F1 or AUC, lowest RMSE, etc.
- In example, it seems that (metric=L1, k=5) performs best

## Model selection + hyperparameter tuning

- **Validation set: choose among competing models**
- Model here consists of combination of **method** and **hyperparameters**
- Three sets: **training**, **validation**, **test**
- In this context, test set sometimes known as the **holdout** set
- Or use **k-fold cross validation** on the training set
- Reason for separate validation set: **DATA LEAKAGE!**
- Keep the test (holdout) set hidden from us until the very end
- **Otherwise, we may unwittingly train on test set! Can fool ourselves.**

## Generalizing grid search

- Now suppose we want to try **lots of ML methods**
- **Each method** has its own **hyperparameters** (differ among methods)
- Some hyperparameters depend on value of other hyperparameters
- Still "grid search" but the "grid" is in higher dimensions (# combos)
- Could write loops, if statements, functions, classes, … ☹
- There are tools to automate this: (sklearn) Pipeline
- But note: grid search **requires exponential time** (# combinations?)
- Ask yourself: How long will it take to try all of these?

# Sklearn Pipeline

There is a learning curve …

# Pipeline

- Metaphor: conveyor belt in a factory
- **Split** data
- **Scale** data (standard or min-max scaler)
- Reduce dimensionality (PCA) [Note: We will see this later in course]
- **Instantiate classifier**
  - classifier = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
- **Fit model to training data**
- **Evaluate predictions on validation set**
- Repeat until we have tried all combinations

## Pipeline (sklearn)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn import svm
```

# Pipeline (sklearn)

```
pipeline_lr=Pipeline([('scalar1',StandardScaler()),
                      ('pca1',PCA(n_components=2)),
                      ('lr_classifier',LogisticRegression())])
pipeline_dt=Pipeline([('scalar2',StandardScaler()),
                      ('pca2',PCA(n_components=2)),
                      ('dt_classifier',DecisionTreeClassifier())])
pipeline_svm = Pipeline([('scalar3', StandardScaler()),
                       ('pca3', PCA(n_components=2)),
                       ('clf', svm.SVC())])
pipeline_knn=Pipeline([('scalar4',StandardScaler()),
                      ('pca4',PCA(n_components=2)),
                      ('knn_classifier',KNeighborsClassifier())])
```

# Pipeline (sklearn)

```
pipelines = [pipeline_lr, pipeline_dt,
             pipeline_randomforest, pipeline_knn]
pipe_dict = {0: 'Logistic Regression', 1: 'Decision Tree',
             2: 'Support Vector Machine',3:'K Nearest Neighbor'}
for pipe in pipelines:
    pipe.fit(X_train, y_train)
for i, model in enumerate(pipelines):
    print("{} Test Accuracy:{}"
          .format(pipe_dict[i],model.score(X_test,y_test)))
```

# Pipeline (sklearn)

```
Logistic Regression Test Accuracy: 0.8666666666666667
Decision Tree Test Accuracy: 0.9111111111111111
Support Vector Machine Test Accuracy: 0.9333333333333333
K Nearest Neighbor Test Accuracy: 0.9111111111111111
```

# Pipeline for hyperparameter tuning

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.ensemble import RandomForestClassifier
pipe = make_pipeline((RandomForestClassifier()))
grid_param = [
{"randomforestclassifier": [RandomForestClassifier()],
 "randomforestclassifier__n_estimators":[10,100,1000],
  "randomforestclassifier__max_depth":[5,8,15,25,30,None],
  "randomforestclassifier__min_samples_leaf":[1,2,5,10,15,100],
  "randomforestclassifier__max_leaf_nodes": [2, 5,10]}]
```

## Pipeline for hyperparameter tuning

```
best_model = gridsearch.fit(X_train,y_train)
best_model.score(X_test,y_test)

OUTPUT - 0.9777777777777777
```

## Reminder

- Advanced tool
- Can also incorporate k-fold CV
- Takes significant time to learn how to use it
- May be useful, but not required
- Doesn't fully automate your work: you still have to think!
- Time required is exponential in number of methods and h-params
- Final caution: BEWARE DATA LEAKAGE!
- Respect the purity of the holdout set

# Occam's Razor

Why less is often more

# Motivation

- "What is the best machine learning method?"
- "Is this a good $R^2$ value?"
- "We got 90% accuracy. Do you think that's good?"
- "Do we have enough data?"
- "We could spend a week and get a really accurate model, but our boss wants the answer now!"
- So, it seems wise to spend some time thinking about models.

# Models

- **Model:** Mathematical representation of a real-world process
- **"All models are wrong, but some are useful"**
  - Attrib. George Box (British statistician) c. 1976
  - But the statement itself is a model …
  - Therefore, it is wrong …
  - but also sometimes useful. ☺
- https://en.wikipedia.org/wiki/All_models_are_wrong

# George Box Quote #1

**2.3 Parsimony**

Since all models are wrong
the scientist **cannot obtain a "correct" one by excessive elaboration.**

On the contrary following William of Occam
[they] should **seek an economical description of natural phenomena.**

Just as the ability to devise **simple but evocative models**
is the signature of the great scientist

so **overelaboration and overparameterization**
is often the mark of mediocrity.

# George Box Quote #2

**2.4 Worrying Selectively**

Since all models are wrong

**the scientist must be alert to what is importantly wrong.**

It is inappropriate to be concerned about mice

when there are tigers abroad.

## George Box Quote #3

Now it would be very remarkable if any system existing in the real world could be *exactly* represented by any simple model. **However, cunningly chosen parsimonious models often do provide remarkably useful approximations.** For example, the law PV = RT relating pressure P, volume V and temperature T of an "ideal" gas via a constant R is not exactly true for any real gas, but it frequently provides a useful approximation …

## George Box Quote #4

All models are approximations.

Assumptions, whether implied or clearly stated, are never exactly true.

***All models are wrong, but some models are useful.***

So the question you need to ask is not "Is the model true?" (it never is)

but **"Is the model good enough for this particular application?"**

## Quote by Statistician David Hand

In general, when building statistical models, we must not forget
**that the aim is to understand something about the real world.**
**Or predict, choose an action, make a decision,**
**summarize evidence, and so on, but always about the real world,**
not an abstract mathematical world: our models are not the reality—
a point well made by George Box in his oft-cited remark that
"all models are wrong, but some are useful."

## Discuss

- **Given competing models, suggest criteria for choosing the best one.**
- **Note: Not just parsimony! Consider the overall utility of the model.**

- Industry problem
- kNN: high accuracy/F1 on test set
- Problem: conveyor belt
- Fast R-CNN: less accurate but more useful

Famous Last Words in Data Science …

# "Our model was 100% accurate."

Possible explanations:

- Problem is very simple
- Data set is very small
- Model is very wrong

# Overfitting

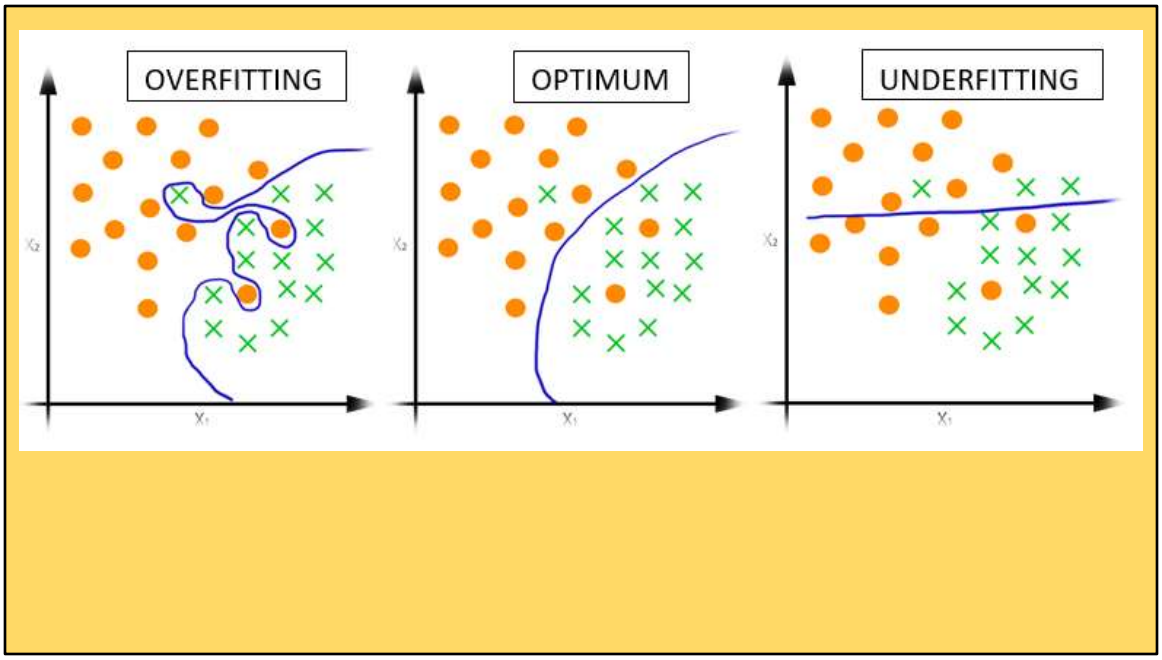
- Modeling error
- Occurs when a function is too closely fit to a limited set of data
- Making an overly complex model to explain idiosyncrasies in the data
- **In reality, most data has some degree of error or random noise**
- Conforming model too closely to slightly inaccurate data can …
  infect model with **substantial errors** and **reduce its predictive power.**

https://www.investopedia.com/terms/o/overfitting.asp

# Underfitting

- Counterpart of overfitting
- Model not complex enough
- Can't accurately capture relationships between features and target
- Problematic or erroneous outcomes on new data
- Often performs poorly even on training data.

https://www.datarobot.com/wiki/underfitting/


OVERFITTING
OPTIMUM
UNDERFITTING

# Bias-Variance Tradeoff

**Analogy: squelch knob on radio**

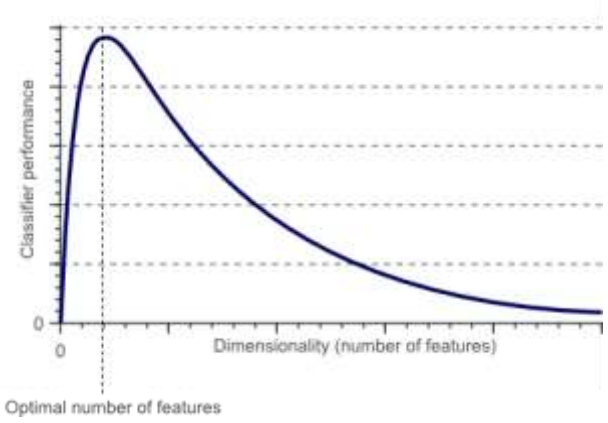
**Bias error**

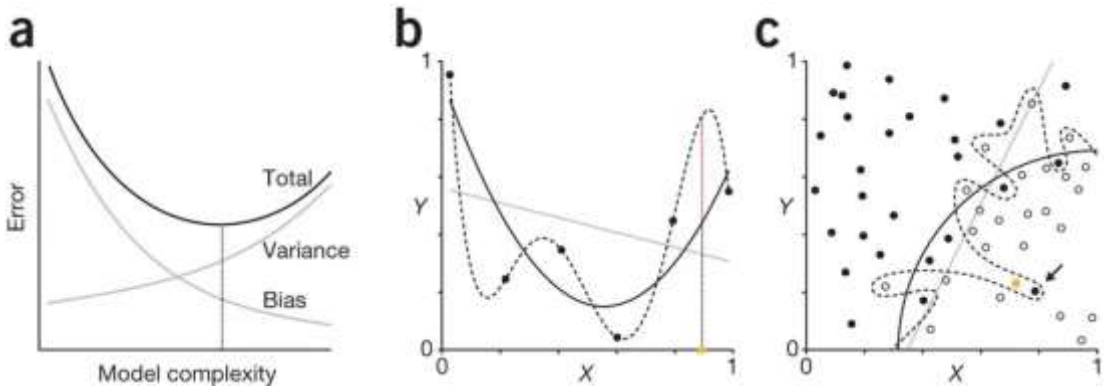
- **"Not enough signal"**
- From erroneous assumptions in the learning algorithm
- Algorithm **misses relevant relations** between features and target outputs
- "Underfitting"

**Variance**

- **"Too much noise"**
- Error from sensitivity to small fluctuations in training set
- Algorithm **models random noise** in training data, rather than intended outputs
- "Overfitting"

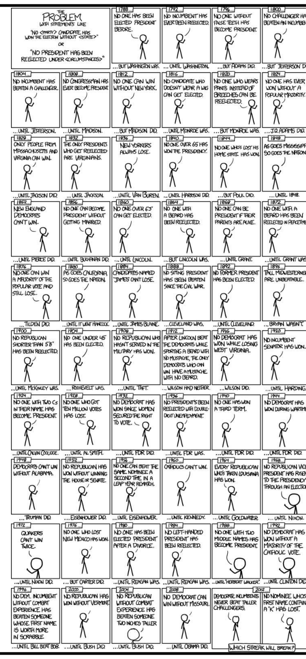
# Curse of Dimensionality



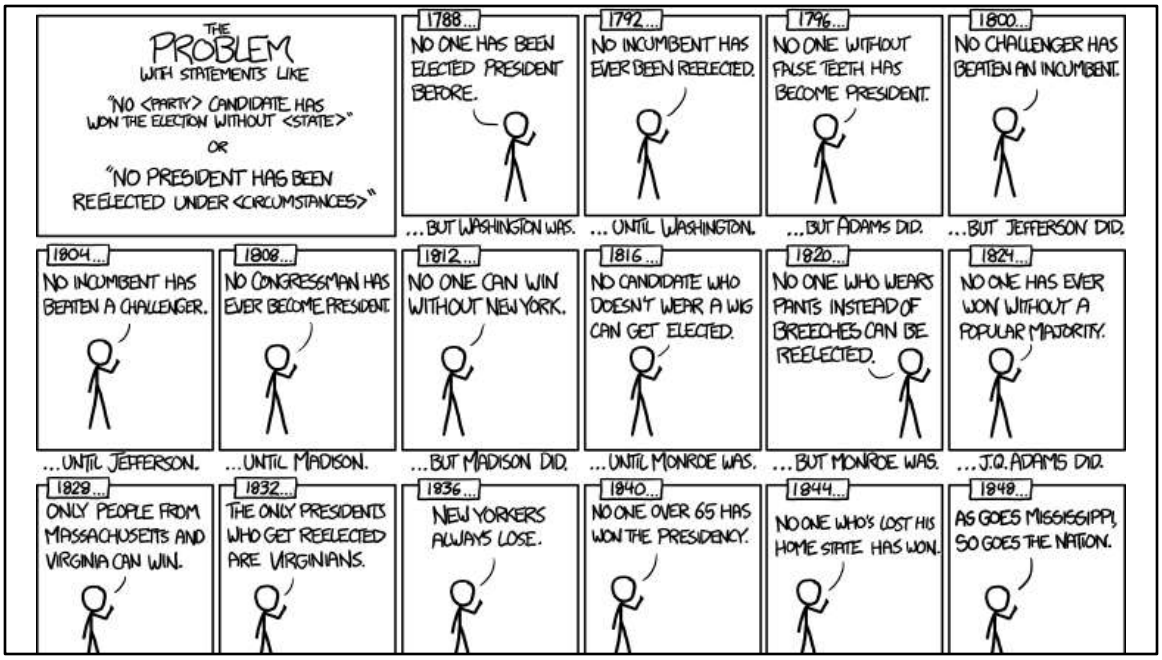


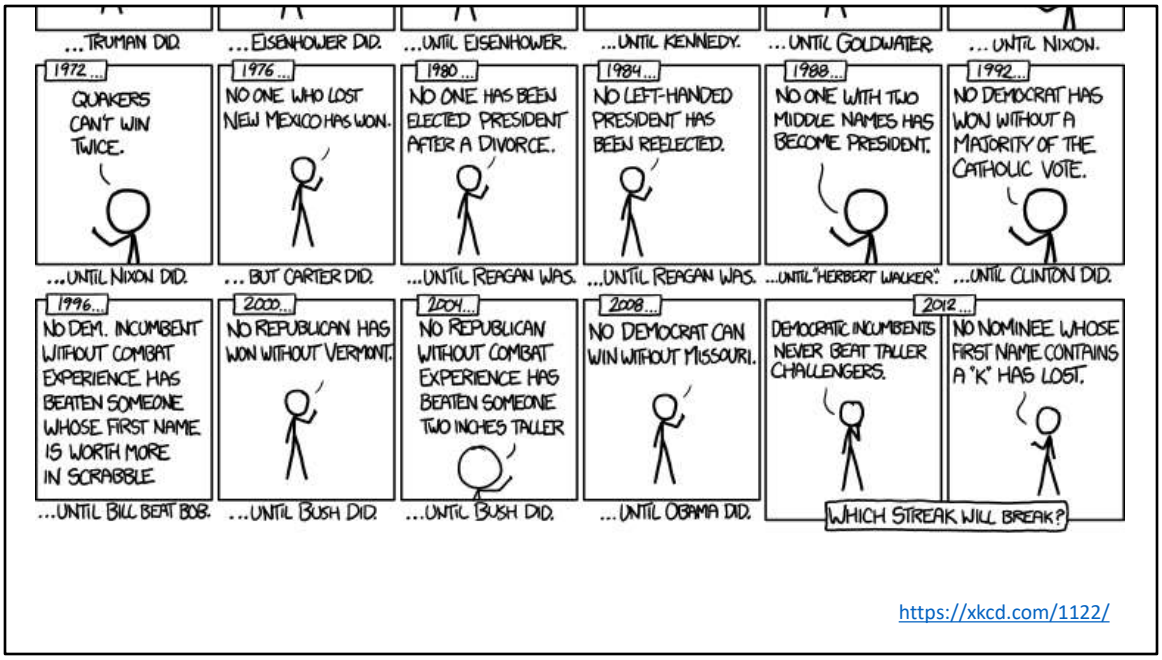

(**a**) When model complexity increases, generally bias decreases and variance increases. The choice of model complexity is informed by the goal of minimizing the total error (dotted vertical line). (**b**) Polynomial fits to data simulated from a third-order polynomial underlying a model with normally distributed noise. The fits shown exemplify underfitting (gray diagonal line, linear fit), reasonable fitting (black curve, third-order polynomial) and overfitting (dashed curve, fifth-order polynomial). There is a large difference (red dotted line) in Y prediction at X = 0.9 (orange circle) between the reasonable and overfitted models. (**c**) Two-class classification (open and solid circles) with underfitted (gray diagonal line), reasonable (black curve) and overfitted (dashed curve) decision boundaries. The overfit is influenced by an outlier (arrow) and would classify the new point (orange circle) as solid, which would probably be an error.

https://www.nature.com/articles/nmeth.3968/figures/1

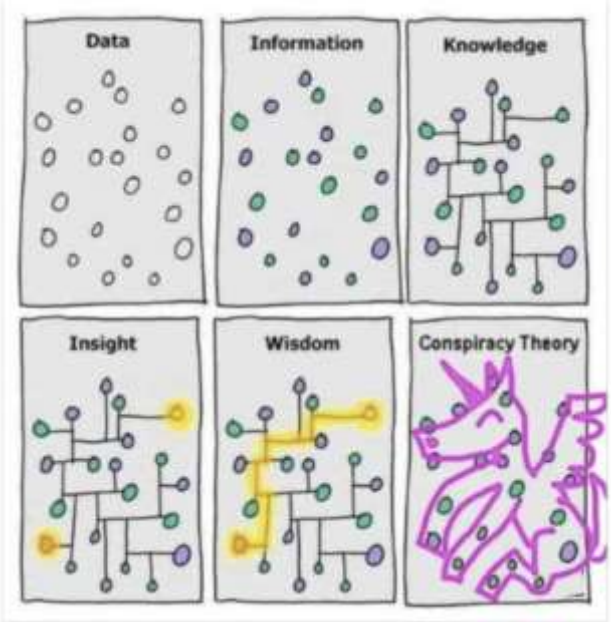



https://xkcd.com/1122/


THE PROBLEM WITH STATEMENTS LIKE
"NO <PARTY> CANDIDATE HAS WON THE ELECTION WITHOUT <STATE>"
OR
"NO PRESIDENT HAS BEEN REELECTED UNDER <CIRCUMSTANCES>"
1788... NO ONE HAS BEEN ELECTED PRESIDENT BEFORE.
...BUT WASHINGTON WAS.
1792... NO INCUMBENT HAS EVER BEEN REELECTED.
... UNTIL WASHINGTON.
1796... NO ONE WITHOUT FALSE TEETH HAS BECOME PRESIDENT.
...BUT ADAMS DID.
1800... NO CHALLENGER HAS BEATEN AN INCUMBENT.
...BUT JEFFERSON DID.
1804... NO INCUMBENT HAS BEATEN A CHALLENGER.
...UNTIL JEFFERSON.
1808... NO CONGRESSMAN HAS EVER BECOME PRESIDENT.
...UNTIL MADISON.
1812... NO ONE CAN WIN WITHOUT NEW YORK.
...BUT MADISON DID.
1816... NO CANDIDATE WHO DOESN'T WEAR A WIG CAN GET ELECTED.
...UNTIL MONROE WAS.
1820... NO ONE WHO WEARS PANTS INSTEAD OF BREECHES CAN BE REELECTED.
...BUT MONROE WAS.
1824... NO ONE HAS EVER WON WITHOUT A POPULAR MAJORITY.
...J.Q. ADAMS DID.
1828... ONLY PEOPLE FROM MASSACHUSETTS AND VIRGINIA CAN WIN.
1832... THE ONLY PRESIDENTS WHO GET REELECTED ARE VIRGINIANS.
1836... NEW YORKERS ALWAYS LOSE.
1840... NO ONE OVER 65 HAS WON THE PRESIDENCY.
1844... NO ONE WHO'S LOST HIS HOME STATE HAS WON.
1848... AS GOES MISSISSIPPI, SO GOES THE NATION.




https://xkcd.com/1122/




Thanks to: Sharon Mega